LEDGEROPS

Finance Calculation Notes

A plain-language reference for the formulas, thresholds, assumptions, and model calculations currently used by LedgerOps.

Purpose

This document explains how dashboard figures, wallet balances, runway forecasts, FX recommendations, anomaly scores, payment-delay risk, and compliance scores are calculated.

It documents the current code behavior. It is not an accounting policy, investment recommendation, credit decision, or regulatory opinion.

Prepared from the LedgerOps codebase
Version date: June 24, 2026

1. Calculation Layers

LedgerOps currently calculates information in three different layers:

Layer	Purpose	Primary data source
Account ledger	Balances, payments, transfers, invoices, and transaction history	Authenticated user or company workspace database records
Dashboard summaries	Fast operational totals and charts	Payments, invoices, transactions, alerts, and FX-rate tables
Intelligence services	Runway, FX, anomaly, payment-delay, and compliance decisions	Account-scoped records transformed into model or rule inputs

Important: The dashboard cash-runway card still uses a lightweight heuristic, while the Cash Runway Intelligence page uses account inflows, outflows, and current cash. These two numbers can differ.

2. Money Storage and Wallet Arithmetic

2.1 Minor-unit storage

Demo wallet balances and chat payment amounts are stored in the currency's minor unit. For INR, USD, and similar currencies, 100 minor units equal one major currency unit.

```
Displayed balance = balance_minor / 100
Stored payment amount = round(displayed amount x 100)
```

A stored balance of `2,500,000` minor units is displayed as `25,000.00`.

2.2 Demo transfer

```
Sender balance after payment = sender balance before payment - payment amount
Recipient balance after payment = recipient balance before payment + payment amount
```

The transfer is rejected when:

- The sender does not have enough balance.
- The sender and recipient demo wallets do not use the same currency.
- The recipient is not a valid demo contact.

Each completed demo transfer creates an outbound transaction for the sender and an inbound transaction for the recipient.

2.3 Card or checkout amount conversion

```
Provider amount_minor = round(invoice amount x 100)
```

When a checkout is verified, LedgerOps checks that the paid amount and paid currency match the invoice before marking the invoice as paid.

3. Dashboard Summary Formulas

3.1 Total transaction volume

```
Total transaction volume = sum(all account Payment.amount values)
```

The implementation sums payment rows belonging to the signed-in account or company workspace. It currently does not filter by payment status, so processing or unsettled payment records may be included if present.

3.2 Pending invoices

```
Pending invoices = count(invoices where status = "pending")
```

3.3 Dashboard cash runway

```
Dashboard runway days = max(21, 82 - 3 x pending invoice count)
```

> This is a display heuristic, not a cash-flow forecast. It should eventually be replaced by the same account-data calculation used by the Cash Runway Intelligence page.

3.4 Currency exposure

```
Exposure[currency] = sum(transaction amounts in currency) + sum(pending invoice amounts in currency)
```

The current dashboard adds all transaction amounts without subtracting outbound transactions. Therefore it measures gross activity plus open receivables, rather than a true signed net currency position.

3.5 Dashboard risk score

```
Anomaly set = transactions where stored risk_score > 65
Dashboard risk score = average(risk_score values in anomaly set)
```

If no transaction exceeds 65, the displayed result is zero.

3.6 Monthly transaction volume

```
Monthly volume[month] = sum(transaction amount for every transaction in that month)
```

Both inbound and outbound transaction amounts are currently added as positive values. The chart therefore represents gross activity, not net cash movement.

3.7 Dashboard cash-flow chart

```
Incoming[month] = sum(all transaction amounts in the month)
Estimated expenses[month] = Incoming[month] x 0.58
```

> The 58% expense value is a placeholder approximation. It is not derived from actual outbound transactions and should not be used for financial decisions.

3.8 Country payment exposure

```
Country volume[country] = sum(transaction amounts associated with that country)
```

4. Cash Runway Intelligence

The intelligence runway uses account activity rather than the dashboard heuristic. It requires at least one incoming and one outgoing transaction. Otherwise LedgerOps returns an insufficient-data message.

4.1 Observation period

```
Observed months = max((latest transaction date - earliest transaction date) / 30, 1)
```

A minimum of one month is used even when the account history covers only a few days. This prevents division by a very small time period, but early estimates remain unstable.

4.2 Observed monthly outgoing

```
Monthly outgoing = sum(outbound transaction amounts) / observed months
```

4.3 Current cash

```
Wallet accounts: current cash = sum(wallet balances)

Other accounts: current cash = max(sum(inbound) - sum(outbound), 0)
```

4.4 Average incoming amount

```
Average incoming = sum(last 12 incoming amounts) / count(last 12 incoming amounts)
```

This is an average per incoming transaction, not a time-normalized monthly revenue value.

4.5 Delayed-invoice drag

```
Delayed-invoice drag = sum(pending invoice amounts) x 0.22
```

The 22% factor represents an assumed collection delay impact. In the current implementation it is only used when current cash is not explicitly provided.

4.6 Monthly burn

```
Monthly burn = expenses + payroll - subscriptions - average incoming
```

The backend currently sends observed monthly outgoing as `expenses`, while payroll and subscriptions are zero until those account data sources are implemented.

4.7 Projected runway

```
Raw runway days = current cash / max(monthly burn, 1) x 30
Projected runway days = integer clamp(raw runway days, 0, 365)
```

Projected days	Risk level
Less than 35	High
35 to 74	Medium
75 or more	Low

4.8 Forecast series

```
Week i cash = current cash - (monthly burn / 4 x i) + (average incoming x 0.2)
```

> **Example: the 24-day result**
> Current cash = INR 8,000
> Observed outgoing = INR 10,050
> Average incoming = INR 50
> Monthly burn = 10,050 - 50 = INR 10,000
> Runway = 8,000 / 10,000 x 30 = 24 days

> A forecast based on only one incoming and two outgoing transactions has low statistical confidence. The current UI labels the method as a forecast, but confidence should eventually reflect sample size and observation length.

5. FX Intelligence

5.1 Selected currency

```
Selected currency = currency with the largest calculated exposure
```

Exposure is built from pending invoices and transaction amounts. The current implementation does not yet consistently remove the account's native currency or subtract outbound amounts.

5.2 Rate return and volatility

```
Rate return[t] = (rate[t] / rate[t-1]) - 1
Raw volatility = standard deviation(rate returns) x 1000
Displayed volatility score = min(raw volatility x 4, 100)
```

Raw volatility	Risk level
Greater than 18	High
Greater than 7 and up to 18	Medium
7 or below	Low

5.3 Trend

```
Trend slope = linear regression slope over up to 30 recent exchange rates
Slope > 0: strengthening; otherwise: weakening
```

5.4 Conversion recommendation

Condition	Hold	Convert now
Strengthening and risk is not High	70%	30%
Medium risk	45%	55%
All other cases	25%	75%

> The recommendation is an operational staging rule, not a prediction that the market will move in a guaranteed direction.

6. Fraud and Anomaly Detection

6.1 Transaction feature vector

```
Features = [amount, country risk, transaction hour, stored transaction risk score]
```

Two unsupervised models assess how far the transaction is from the learned normal pattern: Isolation Forest and One-Class SVM.

6.2 Anomaly score

```
Raw score = 70 - (Isolation Forest decision x 90) - (One-Class SVM decision x 12)
Anomaly score = integer clamp(raw score, 0, 100)
```

Lower model decision values indicate a transaction farther from normal, which raises the displayed anomaly score.

6.3 Explanation rules

Condition	Explanation added
Amount greater than 60,000	Amount is abnormal versus the trained SMB range
Country risk greater than 0.40	Elevated-risk country pattern
Hour before 07:00 or after 22:00	Unusual transaction timing
Counterparty appears once	First-time payer verification
Invoice difference exceeds tolerance	Invoice mismatch

6.4 Invoice mismatch tolerance

```
Tolerance = max(invoice amount x 5%, 100)
Mismatch = absolute(transaction amount - invoice amount) > tolerance
```

6.5 Batch results

```
Average anomaly score = mean(scores for up to 40 recent transactions)
Flagged count = count(transaction scores greater than or equal to 70)
Headline anomaly = highest-scoring transaction in the analyzed batch
```

7. Payment-Delay Risk

The payment-delay feature vector contains:

- log(1 + invoice amount)
- country risk
- currency risk
- average client delay
- payment history
- days until due
- past delay count

7.1 Model blend

```
Delay probability = mean(Logistic Regression probability, Random Forest probability, XGBoost probability)
Delay risk = round(delay probability x 100)
```

7.2 Training label used by the current baseline

```
Synthetic risk = 0.18 + log(1+amount)/24 + country risk + currency risk + history/30 + delays/12 - payment history/2
Training label = delayed when synthetic risk > 0.82
```

> The current baseline models are trained from deterministic synthetic data and persisted after their first training. Predictions use each account's invoice and customer features, but the training set is not yet a production history of real, outcome-labelled invoices.

8. Compliance Score

```
Starting compliance score = 100
Final score = max(100 - applicable deductions, 0)
```

Condition	Deduction
Restricted country: IR, KP, SY, or CU	-45
Transaction amount at least 50,000	-18
KYC document missing	-15
Invoice mismatch beyond max(5%, 100)	-22
Payer name missing	-10

Final score	Status
80 to 100	Pass
55 to 79	Review
0 to 54	Blocked

9. Country and Currency Risk Constants

These constants are features used by the current baseline models.

Country	Risk value	Currency	Risk value
US	0.15	USD	0.12
CA	0.18	CAD	0.18
DE	0.14	EUR	0.16
AE	0.34	AED	0.22
JP	0.38	JPY	0.35
ZA	0.58	ZAR	0.62
BR	0.42	GBP	0.20
IN	0.33	Unknown currency	0.40
Unknown country	0.45		

> These values are engineering constants, not official sovereign-risk ratings. They require governance, documentation, validation, and periodic review before production use.

10. Data Sufficiency Rules

Analysis	Minimum data required
FX	Currency exposure and at least 3 rate observations for the selected currency
Anomaly	At least 1 account transaction
Cash runway	At least 1 incoming and 1 outgoing account transaction
Compliance	At least 1 account transaction with a known payer or counterparty
Payment delay	A selected account invoice and its customer history for meaningful results

When these requirements are not met, the product should show that more account activity is needed rather than inventing a score.

11. Recommended Calculation Improvements

1. Use the account preference currency as the native currency and exclude it from FX exposure.
2. Convert foreign amounts into the native currency before comparing or summing them.
3. Subtract outbound transactions when calculating net exposure and net cash movement.
4. Replace the dashboard runway heuristic with the intelligence runway calculation.
5. Replace the dashboard 58% expense approximation with actual outbound transaction categories.
6. Store payroll, subscriptions, recurring costs, and available bank balances as account data.
7. Calculate confidence from observation length, sample count, missing fields, and model validation.
8. Train production models from outcome-labelled, consented, account-safe historical data.
9. Version every model, formula, threshold, and risk constant used to produce a decision.
10. Keep model recommendations advisory and require human review for high-impact decisions.

12. Code Reference

Area	Current source file
Dashboard totals and charts	`backend/app/routers/resources.py`
Account-scoped intelligence inputs	`backend/app/routers/intelligence.py`
ML formulas and thresholds	`ml-service/app/main.py`
Compliance deductions	`backend/app/services/compliance.py`
Wallet and payment arithmetic	`backend/app/routers/payment_app.py`
Provider minor-unit conversion	`backend/app/services/payment_provider.py`